

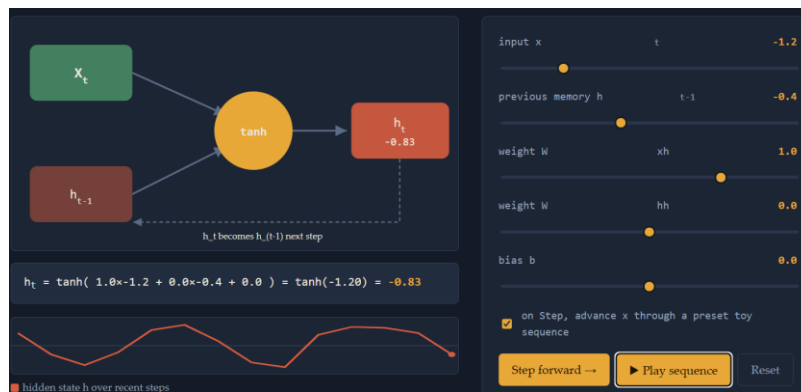
RNN LSTM GRU*Mark Bryan C. Tenebroso*

Provide an explanation of RNN, LSTM, and GRU based on the given .html file. Follow the tasks and instructions provided in the .html file.

RNN

What to try: set W_{hh} near 0 and step a few times — the memory stops carrying forward at all. Then set it near 2 and step again — old memory dominates and the new input barely matters. That balance is the entire vanilla RNN, and it's also why long sequences are hard: there's no separate control for "how much of the past to keep."

$W_{hh} = 0$:

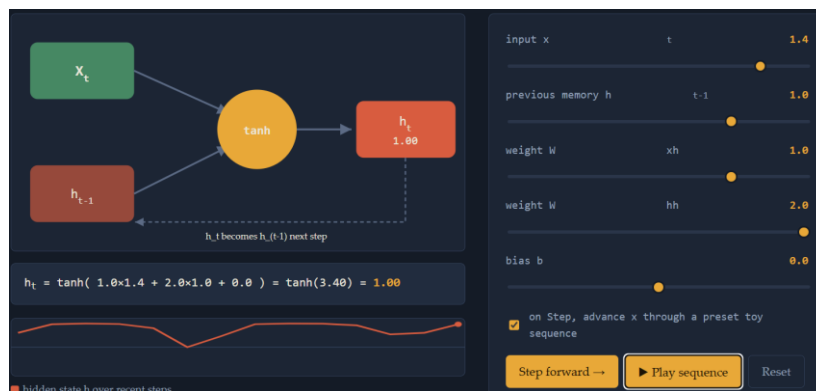


When you turn W_{hh} down near 0, the RNN completely loses its memory.

Usually, the model calculates its new state by mixing new information with old memory. But because the old memory gets multiplied by 0, it vanishes completely. The model now only sees the newest word or input and forgets everything that happened a second ago.

This also stops the model from learning from the past. When it makes a mistake, it tries to look back in time to fix it, but the path to the past is blocked. Because it cannot remember or learn from past steps, it stops acting like a smart sequence reader and behaves like a basic single-step program.

$W_{hh} = 2$:



When you set W_{hh} up near 2, the exact opposite happens: the RNN gets completely trapped in its old memory.

Because the old memory is multiplied by a large factor at each step, it grows larger and larger, completely overwhelming the network. The model gets stuck on past events and almost completely ignores any new input.

This also breaks the model during learning. When it tries to pass error signals back into the past to fix its mistakes, the numbers get multiplied by 2 over and over again. This causes the signals to explode, making the model unstable and completely wild to train.

LSTM

What to try: drag the forget-gate bias to +4 and step through several times — the forget bar stays almost fully open and the cell state barely decays, no matter what x does. Now drag it to -4 — the forget bar slams shut and the cell state resets toward zero every step. That single bias is the difference between "remember for a long time" and "start fresh each step."

Forget-bias = +4



When you set the LSTM's forget-gate bias to +4, you force the memory gate to stay wide open. The forget gate acts like a valve that controls how much of the old memory to keep, using a scale from 0 (forget everything) to 1 (keep everything). Adding +4 to this gate pushes the result close to 1, meaning the gate stays about 98% open at all times.

Because the valve stays wide open, the old memory hardly fades as time passes. The cell state keeps holding onto past information step after step, no matter what new data comes in. This open highway lets the model store important details over long sequences without losing track of what happened at the very beginning

Forget-bias = -4



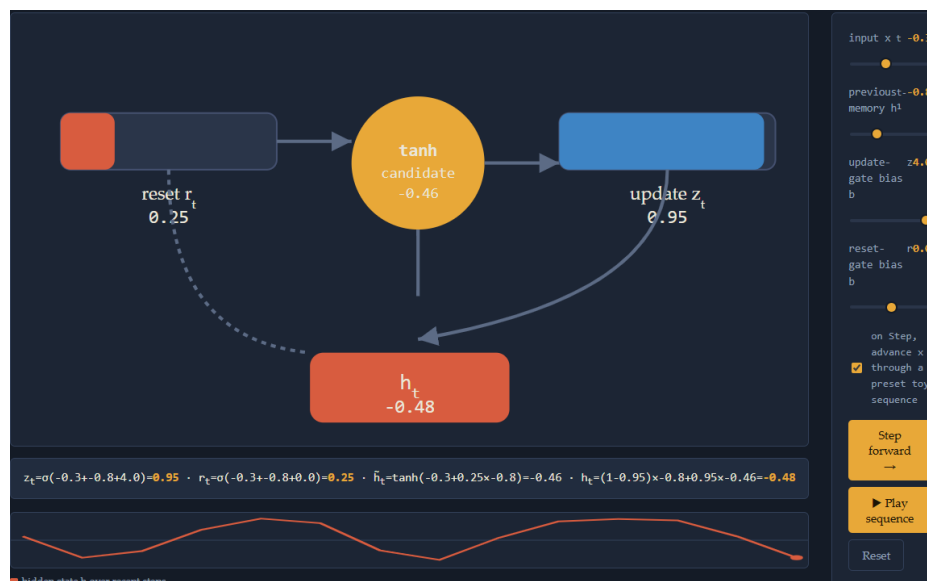
When you turn the LSTM's forget-gate bias down to -4, you are slamming the memory gate completely shut. The forget gate normally decides how much past information to retain, using a scale from 1 (keep everything) to 0 (delete everything). Adding -4 pushes the calculation far into the negative side, forcing the gate to stay near 0 at almost every step.

Because the gate is slammed shut, the old memory gets multiplied by nearly zero and gets wiped out instantly. Every time a new step occurs, the model clears its past cell state and starts fresh, using only the newest input. This causes the LSTM to lose its ability to retain long-term memory, forcing it to focus exclusively on what is happening right now.

GRU

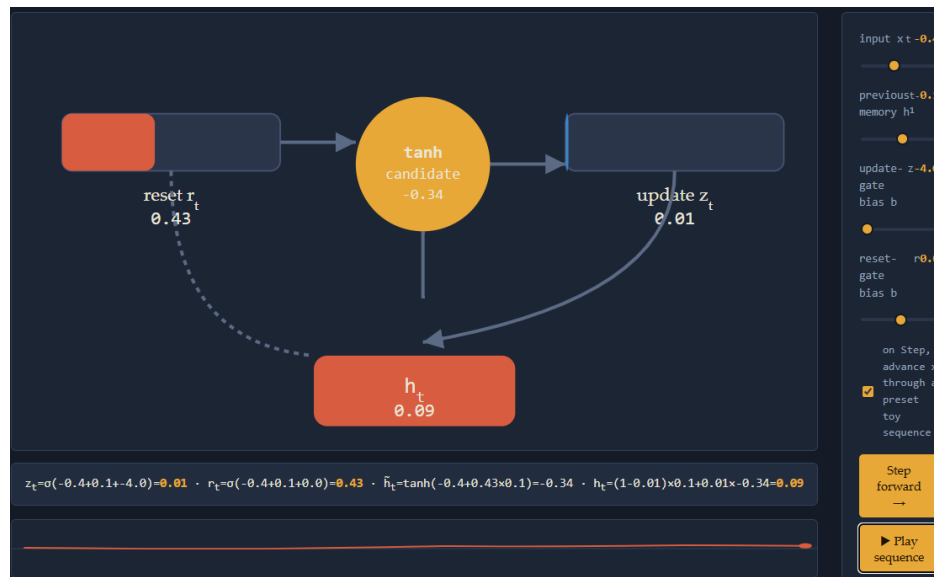
What to try: push the update-gate bias to +4 — z_t saturates near 1, so h_t almost entirely replaces itself with the new candidate every step (short memory). Push it to -4 instead — z_t collapses toward 0, so h_{t-1} just keeps getting carried forward almost untouched (long memory). One gate, one dial, similar effect to LSTM's forget+input pair working together.

update-gate bias = +4



In a GRU, pushing the update-gate bias to +4 forces the update gate z_t to lock open near 1, causing short memory. The update gate works like a single balance scale that decides how much new candidate information to accept versus how much old memory h_{t-1} to retain. When z_t hits 1, the formula completely ignores the old hidden state and replaces it with the newly calculated candidate state at every step. This instantly wipes out past context, forcing the model to operate with a short memory that resets with every new input.

update-gate bias = -4



Pushing the update-gate bias down to -4 creates the exact opposite effect by forcing z_t down to 0, locking in long memory. When z_t collapses near 0, the new candidate state is multiplied by zero and thrown away, while the previous hidden state h_{t-1} is multiplied by 1 and carried forward almost untouched. This transforms the update gate into a highway for long-term memory, allowing historical details to travel across many sequence steps without fading. Instead of using two separate gates like an LSTM's forget and input pair, the GRU uses this single dial to smoothly balance between remembering the past and replacing it with the present.